14. DevSecOps

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 14 of 18 · Thirupathi Keloth · v2.0

Contents

14.0 Why DevSecOps

14.1 Principles

14.2 Security across the delivery lifecycle

14.3 Secrets management

14.4 Static application security testing (SAST)

14.5 Dependencies and software composition analysis (SCA)

14.6 Container security

14.7 Infrastructure as Code security

14.8 Software supply chain security

14.9 Dynamic testing and application security

14.10 Cloud security on AWS

14.11 Kubernetes and runtime security

14.12 Vulnerability management as a process

14.13 Compliance and audit

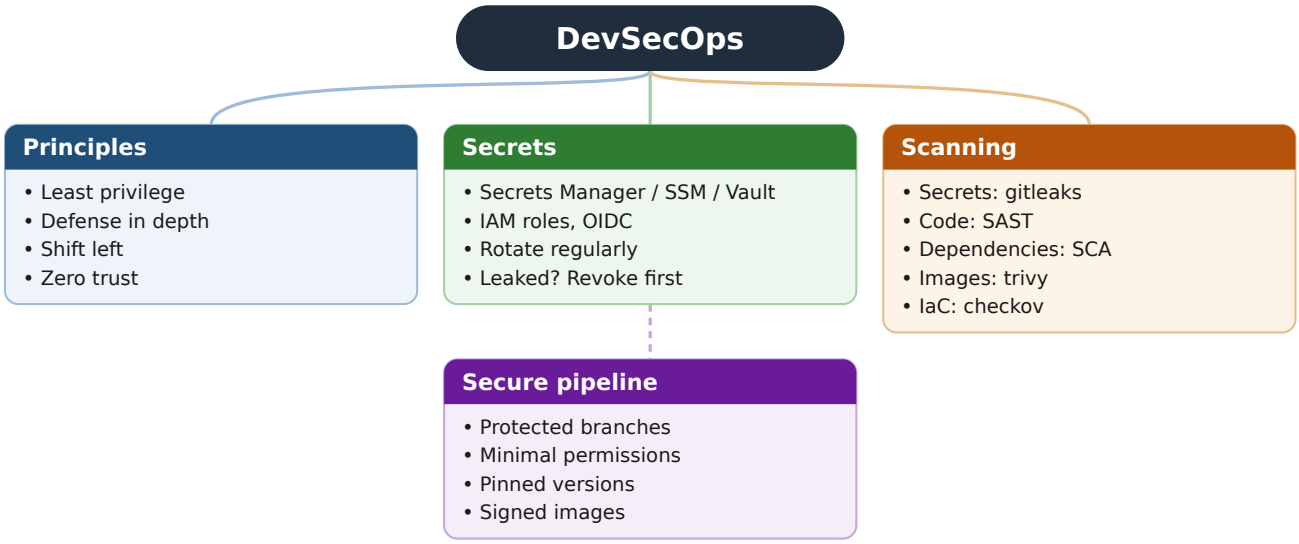
14.14 A complete secure pipeline

14.15 Security incident response for DevOps engineers

14.16 Hands-on labs

14.17 Interview questions

14.18 Self-check



14.0 Why DevSecOps

Traditional security was a gate at the end: a security team reviewed a release weeks after the code was written, found problems that were expensive to fix, and slowed everything down. Modern teams deploy many times a day, so security must be **built into every step**, automated in the pipeline, and owned by the engineers who build the system. That is DevSecOps: **security as code, continuously, by everyone**, with the security team providing tools, guardrails and expertise.

Real breaches DevOps engineers prevent: leaked cloud keys in public repositories (crypto-mining bills within minutes), vulnerable dependencies (Log4Shell, 2021), compromised build systems and packages (SolarWinds 2020, Codecov 2021, the xz backdoor 2024, malicious npm/PyPI packages), public S3 buckets, over-privileged IAM roles, exposed Kubernetes dashboards.

14.1 Principles

Principle	Meaning
Shift left	Find issues as early as possible: IDE, pre-commit, pull request. A bug found in a PR costs minutes; in production, an incident
Shield right	Also protect and detect at runtime: WAF, runtime detection, audit logs, anomaly detection
Defense in depth	Several independent layers (network, identity, workload, data), so one failure is not a breach
Least privilege	Every human, service and pipeline gets only the permissions it needs, for only as long as it needs them
Zero trust	Never trust by network location; authenticate and authorize every request (identity-based access, mTLS)
Secure by default	Templates, modules and golden paths ship with security already on (encryption, private, non-root)
Automate guardrails, not gates	Policies in code that block only real risks, with fast feedback and a documented exception process
Shared responsibility	Cloud provider secures the cloud; you secure what you put in it (IAM, data, network config, OS, apps)

CIA triad (what security protects): **Confidentiality** (only authorized access), **Integrity** (no unauthorized change), **Availability** (accessible when needed).

Threat modeling

Before building, ask four questions (Shostack): *What are we building? What can go wrong? What are we going to do about it? Did we do a good job?* Draw a data-flow diagram with trust boundaries and apply **STRIDE**:

Threat	Violates	Example	Typical control
Spoofing	Authentication	Stolen token used to call the API	MFA, short-lived tokens, mTLS
Tampering	Integrity	Modified container image	Image signing, immutable tags, checksums
Repudiation	Non-repudiation	"I didn't delete that bucket"	CloudTrail / audit logs, signed commits
Information disclosure	Confidentiality	Public S3 bucket, secrets in logs	Encryption, block public access, masking
Denial of service	Availability	Traffic flood, resource exhaustion	WAF, rate limits, autoscaling, quotas
Elevation of privilege	Authorization	Container escape to node, IAM `PassRole` abuse	Non-root, Pod Security, least-privilege IAM

14.2 Security across the delivery lifecycle

```
PLAN            CODE              BUILD / CI               DEPLOY              RUN
threat model,   IDE plugins,      secret scan, SAST,       admission policy,   WAF, runtime detect,
security        pre-commit hooks, SCA, IaC + image scan,   verify signatures,  CSPM, audit logs,
stories         signed commits    SBOM, sign, DAST         least-priv identity rescans, response
```

Category	What it finds	Tools (open source / common)	When
Secret scanning	Keys, tokens, passwords in code and history	gitleaks, TruffleHog, GitHub secret scanning + push protection	Pre-commit, PR, whole history
SAST	Insecure code patterns (injection, unsafe deserialization)	Semgrep, CodeQL, SonarQube, Bandit (Python)	PR
SCA	Known-vulnerable or malicious dependencies, licenses	Dependabot, Renovate, Trivy, Grype, OSV-Scanner, Snyk	PR + daily
IaC scanning	Misconfigurations in Terraform, CloudFormation, K8s YAML, Dockerfiles	Checkov, Trivy (config), KICS, tflint, hadolint, kube-linter	PR
Container image scanning	OS package and library CVEs in images	Trivy, Grype, ECR scanning (Inspector), Docker Scout	Build + registry rescans
SBOM	Inventory of everything inside an artifact	Syft, Trivy, CycloneDX/SPDX formats	Build
Signing / provenance	Proves who built an artifact, from what	cosign (Sigstore), SLSA provenance, GitHub attestations	Build; verify at deploy
Policy as code	Organization rules on config and deployments	OPA/Conftest, Kyverno, Gatekeeper, Sentinel, AWS SCPs	PR + admission
DAST	Vulnerabilities in the running app (black-box)	OWASP ZAP, Burp Suite, Nuclei	Staging
Runtime security	Suspicious behavior in hosts/containers	Falco, GuardDuty, Tetragon	Production
CSPM	Cloud misconfigurations across accounts	AWS Security Hub, Prowler, ScoutSuite	Continuous

Senior advice: start with the highest return (secret scanning, dependency updates, image scanning, IaC scanning, least-privilege IAM), make findings **actionable** (one owner, clear fix), tune false positives aggressively, and fail builds only on what you'll really fix (e.g. critical/high with a fix available).

14.3 Secrets management

Rules

1. **Never** hardcode secrets in code, Dockerfiles, images, Terraform, Ansible, CI files, or Kubernetes YAML in Git.
2. Store them in a **secret manager**; fetch at runtime with an identity (IAM role), not another secret.
3. Prefer **no long-lived secrets at all**: OIDC federation for CI → cloud, IAM roles for workloads, short-lived database credentials.

4. **Rotate** automatically; scope per environment and service.
5. Detect leaks before they land (pre-commit, push protection) and after (history scans).

Store	Notes
AWS Secrets Manager	Versioning, automatic rotation (Lambda; built-in for RDS), resource policies, cross-account; per-secret monthly price
SSM Parameter Store (SecureString)	KMS-encrypted parameters; standard tier free; no built-in rotation
HashiCorp Vault / OpenBao	Dynamic secrets (per-request DB users, AWS keys), PKI, transit encryption, leases
GitHub/GitLab CI secrets	For values the pipeline itself needs (prefer OIDC for cloud)
Kubernetes	External Secrets Operator / CSI driver / Sealed Secrets (Kubernetes notes 12.8)

```
# Create, read and rotate-ready usage of a secret (value generated, never typed in history)
aws secretsmanager create-secret --name prod/shop/db \
  --secret-string "{\"username\":\"shop\",\"password\":\"$(openssl rand -base64 24)\"}"
aws secretsmanager get-secret-value --secret-id prod/shop/db \
  --query SecretString --output text | jq -r .username
```

```
"""Read a secret at runtime with the instance/pod IAM role (no keys in code)."""
import json

import boto3


def get_db_credentials(secret_id: str = "prod/shop/db") -> dict:
    client = boto3.client("secretsmanager")
    response = client.get_secret_value(SecretId=secret_id)
    return json.loads(response["SecretString"])
```

Detecting secrets

```
# .pre-commit-config.yaml: scan staged changes before every commit
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.24.2
    hooks:
      - id: gitleaks
```

```
pip install pre-commit && pre-commit install      # once per clone
gitleaks git --redact -v .                         # scan the whole Git history
gitleaks dir --redact .                            # scan files in a directory
```

Enable **GitHub secret scanning and push protection** on every repository (blocks pushes containing known token formats).

A secret leaked: the playbook

1. **Revoke/rotate immediately.** Assume it was stolen the moment it was pushed (bots scan public GitHub within minutes). Deleting the commit is **not** enough: forks, caches and clones keep it.
2. Check for misuse: CloudTrail for the key's activity (`aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIA...`), billing anomalies, new IAM users/roles, unusual regions.

3. Contain: remove anything created by the attacker, tighten policies.
4. Clean history if needed (`git filter-repo`), but only **after** rotation.
5. Postmortem: why was a long-lived key there at all? Move to OIDC/roles; add pre-commit and push protection.

14.4 Static application security testing (SAST)

SAST analyzes source code without running it. It catches injection (SQL, command), path traversal, unsafe deserialization, weak crypto, hardcoded credentials, SSRF patterns.

```
# VULNERABLE: SQL injection and shell injection
def find_user(cursor, name):
    cursor.execute("SELECT * FROM users WHERE name = '" + name + "'")  # ' OR '1'='1

def ping(host):
    import os
    os.system("ping -c 1 " + host)              # attacker: 8.8.8.8; rm -rf /
```

```
# FIXED: parameterized query, no shell, input validation
import ipaddress
import subprocess


def find_user(cursor, name):
    cursor.execute("SELECT * FROM users WHERE name = %s", (name,))


def ping(host):
    ipaddress.ip_address(host)                      # raises ValueError if not an IP
    subprocess.run(["ping", "-c", "1", host], check=True, timeout=5)
```

```
semgrep scan --config p/python --config p/secrets --error .   # rulesets from the registry
bandit -r src/ -ll                                              # Python, medium+ severity
```

Custom rules encode your organization's lessons:

```
# semgrep-rules/no-shell-true.yml
rules:
  - id: subprocess-shell-true
    languages: [python]
    severity: ERROR
    message: "subprocess with shell=True allows command injection; pass a list instead"
    pattern: subprocess.$FUNC(..., shell=True, ...)
```

CodeQL (GitHub code scanning) builds a database of the code and runs deep data-flow queries (source → sink); free for public repositories, part of GitHub Advanced Security for private ones.

14.5 Dependencies and software composition analysis (SCA)

Modern apps are 80–90% third-party code. SCA matches your dependencies (including transitive ones) against vulnerability databases.

Term	Meaning
CVE	Unique ID for a public vulnerability (`CVE-2021-44228` = Log4Shell)
CVSS	Severity score 0–10 (Critical ≥ 9.0, High 7.0–8.9, Medium 4.0–6.9, Low < 4)

Term	Meaning
EPSS	Probability a CVE will be exploited in the next 30 days
CISA KEV	Catalog of vulnerabilities **known to be exploited**: fix these first
Reachability	Whether your code actually calls the vulnerable function
GHSA / OSV	GitHub advisories / open vulnerability database across ecosystems

Prioritize by risk, not only CVSS: known exploited (KEV) or high EPSS + internet-facing + reachable + fix available = fix now; low-severity in a dev-only tool = backlog.

Keep dependencies current automatically

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: pip
    directory: /
    schedule:
      interval: weekly
    groups:
      minor-and-patch:
        update-types: ["minor", "patch"]   # one PR for all small updates
  - package-ecosystem: docker
    directory: /
    schedule:
      interval: weekly
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
  - package-ecosystem: terraform
    directory: /infra
    schedule:
      interval: weekly
```

Renovate is the more configurable alternative (auto-merge rules, dashboards, monorepos).

Dependency hygiene

- **Lockfiles** committed (`requirements.txt` with pinned versions/hashes, `poetry.lock` , `package-lock.json` , `go.sum`); install with `npm ci` / `pip install --require-hashes` .
- Use a **private proxy registry** (Artifactory, Nexus, CodeArtifact) to cache and control packages.
- Watch for **typosquatting** (`reqeusts` instead of `requests`) and **dependency confusion** (a public package with your internal package's name): scope internal packages, configure registries explicitly.
- Remove unused dependencies: less code, fewer CVEs.
- License compliance: flag copyleft licenses (GPL/AGPL) where policy forbids them.

```
trivy fs --scanners vuln,secret,misconfig --severity HIGH,CRITICAL .
osv-scanner scan source -r .
pip-audit -r requirements.txt
```

14.6 Container security

Layer	Controls
Base image	Minimal (distroless, Chainguard/Wolfi, Alpine, `-slim`), from trusted sources, **pinned by digest**, rebuilt regularly to pick up patches
Build	Multi-stage (no compilers/tools in the final image), no secrets in layers (BuildKit `--mount=type=secret`), `.dockerignore`, non-root `USER`, lint with hadolint
Scan	Trivy/Grype in CI (fail on fixable critical/high), registry rescans for new CVEs (ECR enhanced scanning with Inspector)
Registry	Private, immutable tags, lifecycle policies, access via IAM, pull only from approved registries
Integrity	**Sign** images (cosign), verify signatures at admission
Runtime	Non-root, read-only root FS, drop capabilities, seccomp, no privileged/host mounts, resource limits (Kubernetes 12.5, 12.12)
Detection	Falco/GuardDuty runtime monitoring for shells in containers, unexpected network connections, crypto miners

```
trivy image --severity CRITICAL,HIGH --ignore-unfixed --exit-code 1 shop/api:1.4.2
trivy image --format cyclonedx --output sbom.cdx.json shop/api:1.4.2     # SBOM from the image
syft shop/api:1.4.2 -o spdx-json > sbom.spdx.json                        # alternative SBOM tool
grype sbom:sbom.spdx.json                                                # scan an SBOM later
```

Signing and verifying images (Sigstore cosign)

```
# Keyless signing in CI: identity comes from the CI's OIDC token, recorded in a transparency log
cosign sign --yes 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api@sha256:3f9a...

# Verify: only accept images signed by our repository's release workflow
cosign verify 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api@sha256:3f9a... \
  --certificate-identity-regexp '^https://github.com/acme/shop-api/.github/workflows/release.yml@' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com

# Attach the SBOM as a signed attestation
cosign attest --yes --type cyclonedx --predicate sbom.cdx.json \
  123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/api@sha256:3f9a...
```

Always sign and verify **digests** (`@sha256:...`), not tags: tags can be moved. Enforce verification in the cluster with Kyverno `verifyImages` or the Sigstore policy-controller.

14.7 Infrastructure as Code security

IaC lets you find misconfigurations **before** they exist: public buckets, open security groups, unencrypted storage, wildcard IAM.

```
checkov -d infra/ --framework terraform --compact          # 1000+ built-in checks
trivy config infra/                                        # Terraform, K8s, Dockerfile, Helm
tflint --recursive                                         # provider-aware lint
terraform plan -out tf.plan && terraform show -json tf.plan > plan.json
checkov -f plan.json                                       # scan the plan: resolved values
```

```
# Example findings and their fixes
resource "aws_security_group_rule" "ssh" {
  type              = "ingress"
  from_port         = 22
  to_port           = 22
  protocol          = "tcp"
  cidr_blocks       = ["0.0.0.0/0"] # FINDING: SSH open to the internet -> use SSM Session Manager
  security_group_id = aws_security_group.app.id
}

resource "aws_s3_bucket_public_access_block" "logs" {
  bucket                  = aws_s3_bucket.logs.id
  block_public_acls       = true # FIX: all four settings true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Suppress a finding only with a reason, in code, reviewed like any change: `#checkov:skip=CKV_AWS_144:Cross-region replication not required for dev logs`.

Policy as code with OPA / Conftest

Write your organization's rules once and run them in CI against Terraform plans, Kubernetes YAML, Dockerfiles.

```
# policy/terraform.rego: deny security groups open to the world on non-web ports
package main

import rego.v1

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_security_group_rule"
    rc.change.after.type == "ingress"
    "0.0.0.0/0" in rc.change.after.cidr_blocks
    not rc.change.after.from_port in {80, 443}
    msg := sprintf("%s opens port %v to the internet", [rc.address, rc.change.after.from_port])
}

deny contains msg if {
    rc := input.resource_changes[_]
    rc.type == "aws_s3_bucket"
    not rc.change.after.tags.owner
    msg := sprintf("%s must have an 'owner' tag", [rc.address])
}
```

```
conftest test plan.json --policy policy/        # exit code 1 when any deny rule matches
```

At the cloud-account level, **AWS Service Control Policies** (Organizations) are the hardest guardrail: e.g. deny leaving the organization, deny disabling CloudTrail/GuardDuty, deny regions you don't use. HashiCorp **Sentinel** plays the same role in Terraform Cloud/Enterprise.

14.8 Software supply chain security

Attackers increasingly target the **build path** instead of the app: compromised dependencies, build servers, CI tokens and registries.

```
source ──► build ──► artifact ──► registry ──► deploy
  ▲          ▲          ▲           ▲           ▲
protected  hardened,  SBOM +     immutable,    verify signature
branches,  ephemeral  signature  private,      and provenance
reviews,   runners,   + prove-   scanned       before running
signed     pinned     nance
commits    actions
```

Control	Detail
SLSA framework	Levels for build integrity: provenance exists (L1), generated by a hosted build platform and signed (L2), hardened, isolated builds (L3)
Provenance	Signed statement: which source commit, which workflow, which builder produced this artifact (GitHub artifact attestations, SLSA generator)
SBOM	Inventory (CycloneDX/SPDX) so you can answer "are we affected by the new CVE?" in minutes; required by many customers and regulations
VEX	Statement that a CVE in the SBOM does/doesn't affect the product (reduces noise)
Pin everything	Actions by commit SHA, base images by digest, dependencies by lockfile/hash
CI hardening	Minimal `GITHUB_TOKEN` permissions, OIDC to cloud, environment protection, no untrusted code in privileged workflows, ephemeral runners (CI/CD notes 11.6)
Source protection	Branch protection/rulesets, required reviews, CODEOWNERS, signed commits, MFA for all org members
Scorecards	**OpenSSF Scorecard** rates repositories (yours and dependencies) on these practices

14.9 Dynamic testing and application security

DAST attacks the running application from outside, like an attacker, finding issues that static tools can't see (misconfigured headers, auth flaws, server issues).

```
# OWASP ZAP baseline scan against staging (passive, safe); report as HTML
docker run --rm -v "$PWD:/zap/wrk" -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t https://staging.shop.example.com -r zap-report.html
```

Only scan systems you own or are authorized to test. Full active scans (`zap-full-scan.py`) send attack payloads: staging only.

OWASP Top 10 (2025 edition)

#	Category	DevOps-side controls
A01	Broken Access Control	Authorization on every request, deny by default, tests for IDOR
A02	Security Misconfiguration	IaC scanning, hardened baselines, no default credentials, security headers
A03	Software Supply Chain Failures	SCA, SBOM, signing, pinned dependencies, hardened CI
A04	Cryptographic Failures	TLS everywhere, KMS-managed keys, no weak algorithms, encrypt at rest
A05	Injection	Parameterized queries, input validation, SAST
A06	Insecure Design	Threat modeling, secure design patterns, rate limits

#	Category	DevOps-side controls
A07	Authentication Failures	MFA, SSO/OIDC, session management, credential stuffing protection
A08	Software or Data Integrity Failures	Signed artifacts/updates, verified deserialization, CI integrity
A09	Security Logging and Alerting Failures	Audit logs, centralized logging, alerting on auth anomalies
A10	Mishandling of Exceptional Conditions	Fail securely (closed), no stack traces to users, handle errors consistently

API security: authentication on every endpoint, object-level authorization, rate limiting/throttling (API Gateway, WAF), schema validation, no sensitive data in URLs (they end up in logs).

Web Application Firewall (AWS WAF on ALB/CloudFront/API Gateway): managed rule groups (OWASP core rules, known bad inputs, IP reputation, bot control), rate-based rules. A WAF reduces risk; it doesn't replace fixing the code.

14.10 Cloud security on AWS

Area	Controls
Account structure	AWS Organizations with separate accounts per environment/workload; Control Tower landing zone; dedicated security and log-archive accounts
Identity	IAM Identity Center (SSO) for humans, no IAM users with long-lived keys, MFA everywhere, root account locked away with MFA and no keys
Least privilege	Roles per workload, scoped resources and conditions, IAM Access Analyzer (unused access, policy validation), permission boundaries for delegated admin
Guardrails	SCPs (deny dangerous actions, restrict regions), AWS Config rules (detect non-compliant resources), resource control policies
Logging	CloudTrail organization trail (all regions, log file validation) to a locked S3 bucket in the log-archive account; VPC Flow Logs; ALB/CloudFront/S3 access logs
Detection	**GuardDuty** (threat detection: compromised instances, unusual API calls, crypto mining, EKS/runtime), **Security Hub** (aggregates findings, CIS / AWS Foundational Security Best Practices checks), Inspector (EC2/ECR/Lambda vulnerabilities), Macie (sensitive data in S3), Detective (investigation)
Data protection	KMS encryption at rest (customer-managed keys for sensitive data, key policies), TLS in transit (ACM), S3 Block Public Access at account level, backups with AWS Backup (vault lock for ransomware protection)
Network	Private subnets, security groups as allow lists, no SSH from the internet (SSM Session Manager), VPC endpoints, WAF/Shield on the edge
Response	Automated remediation (EventBridge + Lambda / Config remediation), runbooks for compromised keys and instances

```
prowler aws --compliance cis_3.0_aws        # open-source assessment against CIS benchmarks
aws accessanalyzer list-findings \
  --analyzer-arn arn:aws:access-analyzer:ap-south-1:123456789012:analyzer/org
aws guardduty list-detectors
```

Example: least-privilege policy with conditions

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadShopUploadsOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::acme-shop-uploads-prod/uploads/*"
    },
    {
      "Sid": "DecryptWithShopKeyViaS3Only",
      "Effect": "Allow",
      "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
      "Resource": "arn:aws:kms:ap-south-1:123456789012:key/1111-2222-3333",
      "Condition": {
        "StringEquals": {"kms:ViaService": "s3.ap-south-1.amazonaws.com"}
      }
    }
  ]
}
```

Example SCP guardrail

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyUnusedRegions",
      "Effect": "Deny",
      "NotAction": ["iam:*", "organizations:*", "sts:*", "support:*", "cloudfront:*",
                    "route53:*", "budgets:*", "waf:*", "health:*"],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {"aws:RequestedRegion": ["ap-south-1", "us-east-1"]}
      }
    },
    {
      "Sid": "ProtectSecurityServices",
      "Effect": "Deny",
      "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
                 "guardduty:DeleteDetector", "config:StopConfigurationRecorder"],
      "Resource": "*"
    }
  ]
}
```

14.11 Kubernetes and runtime security

Kubernetes controls are covered in the Kubernetes notes (RBAC, Pod Security, NetworkPolicy, admission policies, secrets: 12.8, 12.12). Runtime detection adds the last layer: **Falco** watches system calls via eBPF and alerts on suspicious behavior.

```
# Falco custom rule: alert when someone opens a shell in a production container
- rule: Shell in production container
  desc: Interactive shell started in a container in a prod namespace
  condition: >
    spawned_process and container and proc.name in (bash, sh, zsh)
    and proc.tty != 0 and k8s.ns.name startswith "prod"
  output: >
    Shell in prod container (user=%user.name pod=%k8s.pod.name ns=%k8s.ns.name
    cmd=%proc.cmdline image=%container.image.repository)
  priority: WARNING
  tags: [container, shell, mitre_execution]
```

Other runtime tools: Tetragon (eBPF enforcement), GuardDuty EKS Runtime Monitoring, KubeArmor. Audit: **kube-bench** (CIS), Kubescape, Trivy `k8s` scans.

14.12 Vulnerability management as a process

Scanners produce thousands of findings; the process decides what gets fixed.

Step	Practice
Inventory	Know every asset: images in use (not just built), hosts, functions, dependencies (SBOMs)
Detect	Continuous scanning in CI **and** of running workloads/registries (new CVEs appear for old images)
Prioritize	Severity + exploitability (KEV, EPSS) + exposure (internet-facing?) + reachability + data sensitivity
Assign	Route to the owning team automatically (tags, CODEOWNERS, service catalog)
Fix within SLA	Example: critical 7 days, high 30 days, medium 90 days; KEV-listed critical internet-facing: 48 hours
Exceptions	Time-limited, approved risk acceptance with compensating controls, recorded (e.g. `.trivyignore` entries with expiry and ticket)
Measure	Open findings by age/severity, % within SLA, mean time to remediate, base image freshness

The best fix is often **rebuilding on a fresh base image**: automate weekly rebuilds and redeploys so patching is routine, not a project.

14.13 Compliance and audit

Framework	What it is	DevOps evidence
SOC 2	US audit of security/availability/confidentiality controls	Access reviews, change management (PRs, approvals), monitoring, incident process, backups
ISO/IEC 27001	International information security management standard	Risk register, policies, asset inventory, controls evidence
PCI DSS	Card payment data	Network segmentation, encryption, vulnerability scans, logging, quarterly reviews
HIPAA	US health data	Encryption, access logs, BAAs
GDPR / India DPDP Act 2023	Personal data protection	Data minimization, consent, retention limits, breach notification, data residency

Framework	What it is	DevOps evidence
CIS Benchmarks	Hardening baselines for OS, cloud, Kubernetes	Automated benchmark scans

Compliance as code: controls implemented as policies (SCPs, Config rules, Kyverno, OPA), evidence collected automatically (CloudTrail, Git history, pipeline logs, AWS Audit Manager), continuous instead of yearly. Git-based change management (PR review + CI + automated deploy) is itself strong audit evidence.

14.14 A complete secure pipeline

All controls together in GitHub Actions. Cheap checks run in parallel on every PR; the image is built, scanned, given an SBOM, pushed, signed and attested only on `main`.

```yaml
name: secure-delivery
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

env:
  REGISTRY: 123456789012.dkr.ecr.ap-south-1.amazonaws.com
  IMAGE: shop/api

jobs:
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0                 # full history for secret scanning
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

  sast:
    runs-on: ubuntu-latest
    container:
      image: semgrep/semgrep:1.95.0
    steps:
      - uses: actions/checkout@v4
      - run: semgrep scan --config p/python --config semgrep-rules/ --error

  dependencies-and-iac:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aquasecurity/trivy-action@0.28.0
        with:
          scan-type: fs
          scanners: vuln,misconfig
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      - name: Policy checks on Kubernetes manifests
        env:
          VER: "0.56.0"
        run: |
          BASE=https://github.com/open-policy-agent/conftest/releases/download
          curl -sSL -o conftest.tgz "${BASE}/v${VER}/conftest_${VER}_Linux_x86_64.tar.gz"
          tar xzf conftest.tgz conftest
          ./conftest test k8s/ --policy policy/k8s/

  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest -q

  image:
    needs: [secrets, sast, dependencies-and-iac, tests]
```

```
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write                    # OIDC for AWS and keyless signing
      attestations: write
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-ecr-push
          aws-region: ap-south-1
      - uses: aws-actions/amazon-ecr-login@v2
      - uses: docker/setup-buildx-action@v3
      - id: build
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE }}:sha-${{ github.sha }}
          sbom: true                     # BuildKit SBOM attestation
          provenance: mode=max           # BuildKit SLSA provenance
      - name: Scan the pushed image
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.REGISTRY }}/${{ env.IMAGE }}@${{ steps.build.outputs.digest }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      - uses: sigstore/cosign-installer@v3
      - name: Sign the image digest (keyless)
        env:
          DIGEST: ${{ steps.build.outputs.digest }}
        run: cosign sign --yes "${REGISTRY}/${IMAGE}@${DIGEST}"
      - uses: actions/attest-build-provenance@v2
        with:
          subject-name: ${{ env.REGISTRY }}/${{ env.IMAGE }}
          subject-digest: ${{ steps.build.outputs.digest }}
          push-to-registry: true
```

Then: the GitOps step bumps the **digest** in the config repo (CI/CD notes 11.6); Kyverno in the cluster verifies the cosign signature before admitting the pod; DAST runs against staging after deployment; production deploys need approval and pass the same verification.

Pipeline hygiene: pin third-party actions to commit SHAs in real repositories (tags shown here for readability), keep the scanners' versions pinned and updated by Dependabot, and publish results as SARIF to the code-scanning tab so developers see findings in the PR.

14.15 Security incident response for DevOps engineers

Scenario	First actions
Leaked AWS access key	Deactivate/delete the key → CloudTrail review of its activity → remove attacker resources (check all regions: EC2, Lambda, IAM users/roles/keys) → billing check → postmortem and move to roles/OIDC
Compromised EC2 instance (GuardDuty finding)	Isolate (replace security group with a deny-all/forensics SG), **don't terminate** before snapshotting EBS volumes and capturing memory if needed → revoke the instance role sessions → rebuild from a clean image
Malicious dependency published	Check SBOMs: which artifacts include it → pin to a safe version → rebuild/redeploy → rotate secrets the build or app could reach

Scenario	First actions
Exposed S3 bucket	Enable Block Public Access → review access logs/CloudTrail data events for downloads → assess data exposed → legal/privacy notification process
Suspicious shell in a pod (Falco)	Capture details (pod spec, image digest, events) → isolate with a deny-all NetworkPolicy → snapshot → delete and redeploy → investigate image and entry point

Principles: **contain first, preserve evidence, rotate credentials, rebuild from known-good, then learn**. Involve the security team early; follow your organization's legal/notification process for data incidents.

14.16 Hands-on labs

#	Lab
1	Install pre-commit + gitleaks in a test repo; commit a fake key (`AKIAIOSFODNN7EXAMPLE` is AWS's documented example) and watch it get blocked
2	Run `gitleaks git` over one of your repositories' full history
3	Write the vulnerable Python from 14.4; scan with Semgrep and Bandit; fix it and re-scan; add the custom rule
4	Add Dependabot to your GitHub repo; review and merge the PRs it opens
5	Scan an old image (`python:3.8`) vs `python:3.12-slim` vs a distroless image with Trivy; compare CVE counts
6	Generate an SBOM with Syft; then scan the SBOM with Grype
7	Sign an image in a local registry (`registry:2`) with a cosign key pair; verify it; tamper (push a new image to the same tag) and see verification by digest protect you
8	Run Checkov and Trivy config on your Terraform project; fix two findings; skip one with a written reason
9	Write the Conftest policy from 14.7; test it against a Terraform plan JSON
10	Run the ZAP baseline scan against a local app (e.g. OWASP Juice Shop in Docker on your own machine)
11	Build the 14.14 pipeline in your GitHub repo (without the AWS parts at first: push to GHCR instead)
12	(AWS, mostly free within limits; check pricing) Enable GuardDuty and Security Hub in a lab account for a few days, read the findings, then disable them

14.17 Interview questions

1. What is DevSecOps, and how is it different from a security review at the end?
2. Shift left vs shield right; give tool examples for each.
3. Walk through threat modeling with STRIDE for a simple web app on AWS.
4. Where do secrets belong? How do CI pipelines get cloud access without stored keys?
5. A developer pushed an AWS key to a public repository. What do you do, in order?
6. SAST vs DAST vs SCA vs IaC scanning: what does each find, and where in the pipeline?
7. CVE, CVSS, EPSS, KEV: how do you prioritize hundreds of vulnerabilities?
8. How do you keep dependencies and base images patched continuously?
9. How do you secure a Docker image from build to runtime?
10. What is an SBOM and why do organizations require it?
11. Explain image signing with cosign; why sign digests instead of tags?

12. What is SLSA? What is build provenance?
13. Describe supply-chain attacks you know (SolarWinds, Codecov, xz) and the controls against them.
14. How would you prevent a Terraform change that opens SSH to the world from being applied?
15. What is policy as code? OPA/Conftest vs Kyverno vs SCPs.
16. Name OWASP Top 10 categories and a DevOps control for three of them.
17. How do you secure GitHub Actions workflows?
18. AWS security services: GuardDuty vs Security Hub vs Inspector vs Config vs CloudTrail vs Macie.
19. How do you design least-privilege IAM? What tools help?
20. Design guardrails for a multi-account AWS organization.
21. How do you handle a critical CVE in a base image used by 50 services?
22. What is runtime security (Falco), and what would you alert on?
23. How do you balance security gates against developer speed (false positives, exceptions)?
24. What evidence does DevOps provide for a SOC 2 or ISO 27001 audit?

14.18 Self-check

- [ ] I can explain DevSecOps principles, the CIA triad and STRIDE threat modeling.
- [ ] I can place secret scanning, SAST, SCA, IaC scanning, image scanning, SBOM, signing, DAST and runtime detection in the right pipeline stage.
- [ ] I never store secrets in code and can set up a secret manager, OIDC and leak detection.
- [ ] I can run the leaked-credential playbook from memory.
- [ ] I can prioritize vulnerabilities using severity, exploitability, exposure and reachability.
- [ ] I can harden, scan, sign and verify container images.
- [ ] I can scan Terraform with Checkov/Trivy and write an OPA policy.
- [ ] I can explain supply chain security (SLSA, provenance, pinning, CI hardening).
- [ ] I can describe the AWS security service stack and multi-account guardrails.
- [ ] I can build the complete secure pipeline in 14.14.

REMEMBER

- Never in Git: **passwords, keys, tokens, .env files, tfstate**.
- Security is every layer: IAM → network → server → container → Kubernetes → data.